

SLAM - Ex 3

Iris Kaplan & William Ayoub

May 23, 2026

This document contains the answers to VAN Exercise 3. The code for this exercise is in [this link](#). The final version used for this submission corresponds to commit `9a4d579`.

1 Creating Point Clouds for Two Stereo Pairs

We reused the stereo pipeline from Exercise 2 to create 3D point clouds for two consecutive stereo pairs. For each stereo pair, we matched features between the left and right images, removed stereo outliers using the vertical-deviation criterion, and triangulated the remaining matches.

For Exercise 3, we use a stricter version of the stereo pipeline. In addition to the vertical-deviation test, we apply Lowe's ratio test to the feature matches and remove triangulated points with invalid depth, namely points behind the camera or points whose depth is larger than a fixed threshold. This filtering is useful for the following PnP and RANSAC steps, since erroneous 3D points can strongly affect the estimated camera motion.

The first point cloud was created from stereo pair 0, as in Exercise 2. The second point cloud was created from the next stereo pair:

`image_0/000001.png`, `image_1/000001.png`.

The resulting point clouds are shown in Figure 1.

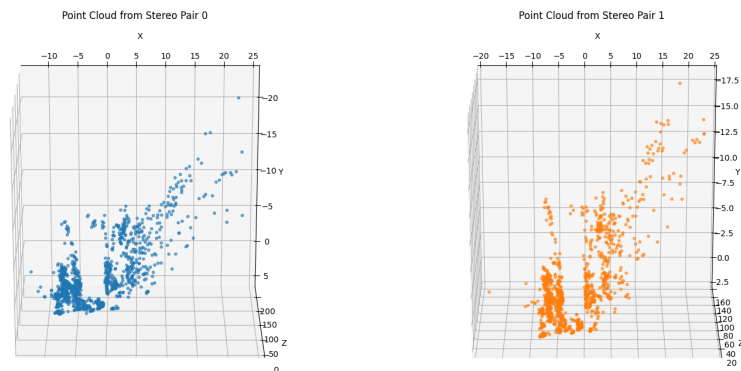


Figure 1: 3D point clouds obtained from stereo pairs 0 and 1 after stereo outlier rejection and depth filtering.

In the implementation of Exercise 3, we use ORB features instead of SIFT features because they are faster and worked better for tracking over the full sequence. For backward compatibility, the code still supports the SIFT-based pipeline used in the previous exercises.

2 Feature Matching Between Consecutive Frames

As in Exercise 1, we matched features using OpenCV. In addition to the stereo matches between left and right images at the same frame, we added temporal matching from $left_0$ to $left_1$. We did not apply the vertical-deviation test to these matches, but we rejected outliers with Lowe ratio test.

The resulting matches are shown in Figure 2. Stereo matches are shown in blue, and the temporal matches between the two left images are shown in red.

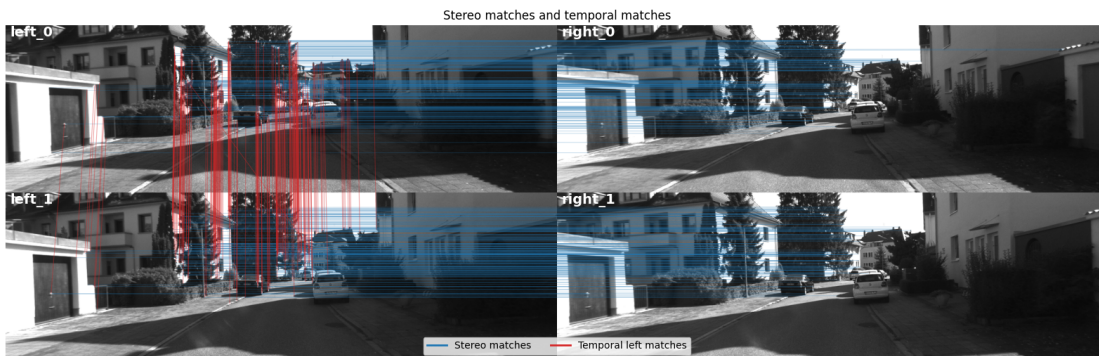


Figure 2: Feature matches across the first two stereo pairs. The top row shows `left_0` and `right_0`, and the bottom row shows `left_1` and `right_1`. Blue lines show stereo matches between the left and right images of each frame. Red lines show temporal matches between `left_0` and `left_1`.

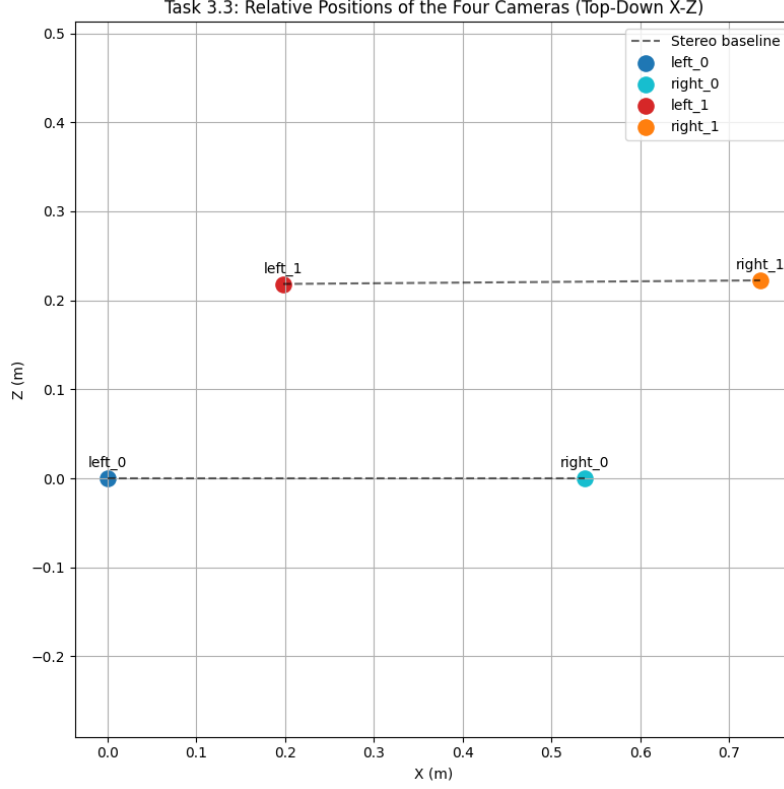


Figure 3: Relative positions of the four cameras viewed from above. The camera `left_0` is used as the reference camera.

3 Pose Estimation Using PnP

We estimated the pose of the left camera in frame 1 relative to the left camera in frame 0. For this step, we used keypoints that were observed in all four images: `left_0`, `right_0`, `left_1`, and `right_1`.

The stereo pair (`left_0`, `right_0`) gives a triangulated 3D point $X_i^{(0)}$, expressed in the coordinate system of `left_0`. The temporal match between `left_0` and `left_1` gives the corresponding 2D image location $u_i^{(1)}$ in `left_1`. Thus, we obtain 3D–2D correspondences

$$X_i^{(0)} \in \mathbb{R}^3 \quad \leftrightarrow \quad u_i^{(1)} \in \mathbb{R}^2.$$

In our implementation, we found 566 keypoints that were matched across all four images and could therefore be used as valid 3D-2D correspondences for PnP. Using four such correspondences, we applied OpenCV’s PnP implementation to estimate an extrinsic matrix $[R \mid t]$, where $R \in \mathbb{R}^{3 \times 3}$ and $t \in \mathbb{R}^{3 \times 1}$. This matrix defines the transformation from `left_0` coordinates to `left_1` coordinates:

$$X^{(1)} = RX^{(0)} + t.$$

3.1 Transformation Composition

If a transformation is defined as $T(x) = Rx + t$, we can express the composition of two transformations, $T_{A \rightarrow B}(x) = R_1x + t_1$ and $T_{B \rightarrow C}(x) = R_2x + t_2$, as:

$$T_{A \rightarrow C}(x) = R_2(R_1x + t_1) + t_2 = R_2R_1x + R_2t_1 + t_2.$$

Therefore, the relative rotation and translation are:

$$R_{A \rightarrow C} = R_2R_1, \quad t_{A \rightarrow C} = R_2t_1 + t_2.$$

To implement this efficiently in code, we represent transformations using homogeneous coordinates. The Perspective-n-Point (PnP) algorithm yields a 3×4 transformation matrix $[R \mid t]$. By appending a row $[0 \ 0 \ 0 \ 1]$, we construct a 4×4 homogeneous transformation matrix T :

$$T = \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix}$$

In this 4×4 form, transformation composition simplifies entirely to standard matrix multiplication. To find the transformation from A to C , we simply multiply $T_{B \rightarrow C}$ by $T_{A \rightarrow B}$:

$$T_{A \rightarrow C} = T_{B \rightarrow C}T_{A \rightarrow B} = \begin{bmatrix} R_2 & t_2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R_1 & t_1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} R_2R_1 & R_2t_1 + t_2 \\ 0 & 1 \end{bmatrix}$$

Notice that the top three rows of the resulting matrix exactly match the manually derived $R_{A \rightarrow C}$ and $t_{A \rightarrow C}$. This property allows the visual odometry pipeline to accumulate the camera's global trajectory by continuously matrix-multiplying the new relative transformation step with the accumulated state:

$$T_{\text{global_new}} = T_{\text{step}} T_{\text{global_old}}$$

3.2 Camera Location in Global Coordinates

The extrinsic matrix $[R \mid t]$ maps a point from the global coordinate system to the camera coordinate system:

$$X_{\text{cam}} = RX_{\text{global}} + t.$$

The camera location in global coordinates is the point C whose coordinates in the camera frame are the origin. Therefore, we require

$$[R \mid t] \begin{bmatrix} C \\ 1 \end{bmatrix} = 0,$$

or equivalently

$$RC + t = 0.$$

Thus, the camera center in the global coordinate system is

$$C = -R^{-1}t = -R^T t,$$

since R is a rotation matrix.

3.3 Relative Camera Positions

We used `left_0` as the global reference frame. Therefore,

$$C_{\text{left}_0} = 0.$$

The location of `right_0` is determined from the stereo baseline in the calibration. From the PnP result $[R \mid t]$, the location of `left_1` in the `left_0` coordinate system is

$$C_{\text{left}_1} = -R^T t.$$

Finally, `right_1` is obtained by applying the same stereo baseline relative to `left_1`. These four camera centers are plotted from above in Figure 3.

4 Outlier Rejection

We evaluated the transformation T estimated in the previous section by checking its consistency with all four-view matches. For each 3D point triangulated from (`left_0`, `right_0`), we projected the point onto `left_0`, `right_0`, `left_1`, and `right_1`. A point was defined as a *supporter* if its projection was within 2 pixels of the matched location in all four images.

Out of 566 four-view matches, 194 were supporters of the estimated transformation, giving a supporter ratio of 34.28%. The matches on `left_0` and `left_1` are shown in Figure 4, with supporters highlighted in a different color.

Task 3.4: Matches and Supporters of the PnP Transformation



Figure 4: Matches between `left_0` and `left_1`. All matches are shown in cyan, and supporters of the estimated transformation T , using a 2-pixel reprojection threshold, are highlighted in orange.

5 RANSAC

We implemented RANSAC with PnP as the inner model. In each iteration, we sampled four 3D-2D correspondences, estimated a candidate transformation T , and counted its supporters



Figure 5: RANSAC inliers and outliers on `left_0` and `left_1`.

using the 2-pixel reprojection threshold. The transformation with the largest supporter set was selected, and then refined by recomputing PnP using all inliers.

In the end of the iterations, RANSAC found 457 inliers and 109 outliers, corresponding to an inlier ratio of 80.74%. This is shown in Figure 5.

Using the refined transformation, we transformed the point cloud of pair 0 into the coordinate system of pair 1. The two point clouds are shown from above in Figure 6.

6 Full Trajectory

We applied the tracking pipeline to the entire sequence, containing 3299 frames. For each consecutive pair of frames, we estimated the relative transformation using temporal feature matching, stereo triangulation, and RANSAC with PnP. We then composed these relative transformations to estimate the poses of all left cameras in the coordinate system of `left_0`.

The full tracking run took approximately 5.76 minutes after using ORB features. The estimated camera trajectory is shown in Figure 7, together with the ground-truth trajectory.

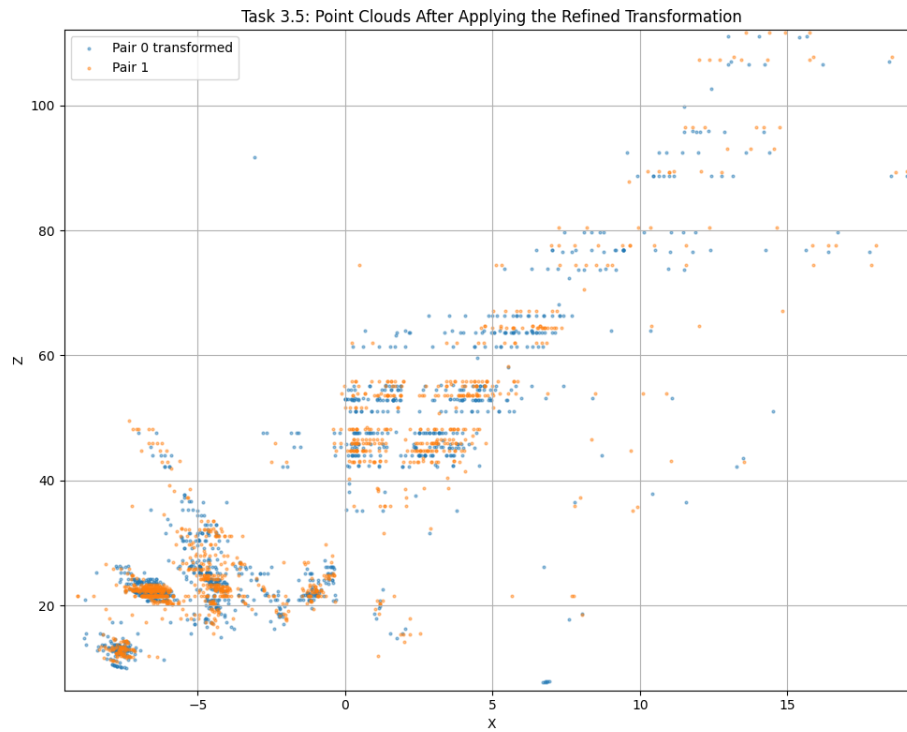


Figure 6: Top-down view of pair 1 and pair 0 after applying the refined transformation.

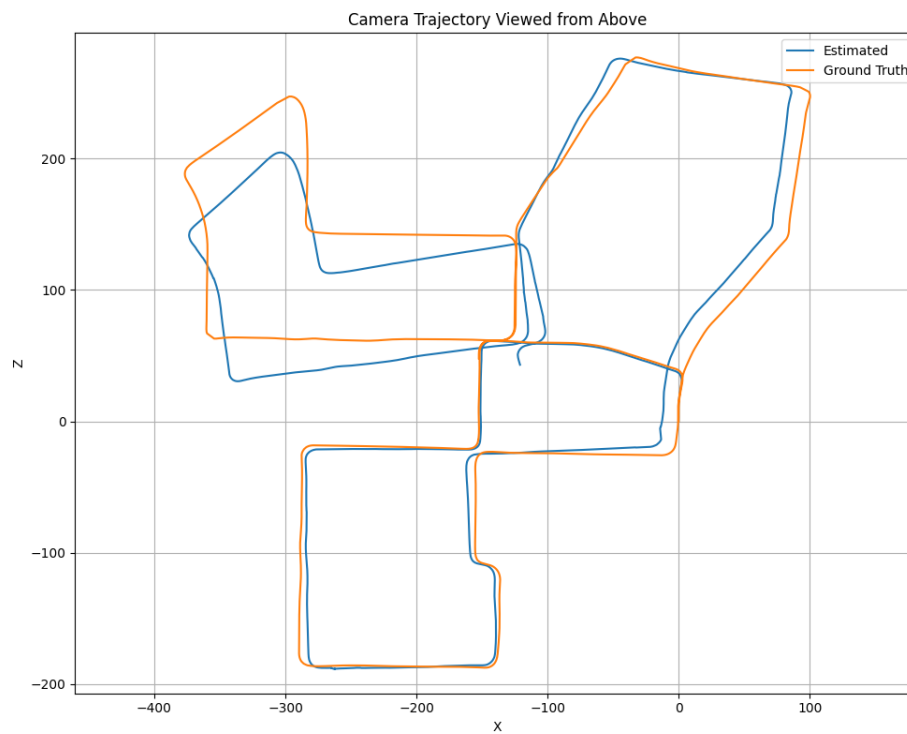


Figure 7: Estimated trajectory of the left camera compared to the ground-truth trajectory, viewed from above in the X - Z plane.